



Python Programming Essentials

A Practical Introduction to Python for Beginners

Why Python?



Simple & Readable

Python reads like English.
Designed for clarity over
complexity.



#1 Language Worldwide

Most popular language
for data, AI, cloud & automation.



Massive Ecosystem

300,000+ packages on PyPI.
A library for everything.

|| *Life is short, use Python.* ||

Setting Up Python

>_ Installation

- 1 Download from python.org
- 2 Check "Add Python to PATH"
- 3 Install (click Next through wizard)
- 4 Verify: `python --version`



Recommended Tools

- ✓ **VS Code** — Lightweight code editor with Python extension
- ✓ **Jupyter Notebook** — Interactive coding environment (great for learning)
- ✓ **Python REPL** — Built-in interactive shell — type `python` in terminal
- ✓ **pip** — Package manager — install libraries instantly

Variables & Data Types

>>> Code Example

```
# Variables – no type declaration needed
name = "Azure"
version = 3.12
is_active = True
count = 42

# Check types
print(type(name))    # <class 'str'>
print(type(version)) # <class 'float'>
```

Core Data Types

str "Hello"
Text / strings

int 42
Whole numbers

float 3.14
Decimal numbers

bool True / False
Boolean values

Strings & f-Strings

```
>>> f-Strings – The Modern Way to Format Text
```

```
# Basic string operations
```

```
greeting = "Hello, World!"
```

```
print(greeting.upper())      # HELLO, WORLD!
```

```
print(greeting.lower())     # hello, world!
```

```
print(len(greeting))        # 13
```

```
# f-Strings (formatted string literals) – Python 3.6+
```

```
name = "Cloud"
```

```
version = 3
```

```
print(f"Welcome to {name} v{version}!")
```

```
# Output: Welcome to Cloud v3!
```

💡 *f-strings are the most common pattern — you'll see them in almost every Python script.*

Lists & Dictionaries

Lists []

Ordered, mutable collection

```
fruits = ["apple", "banana", "cherry"]
```

Access by index (starts at 0)

```
print(fruits[0])    # apple
```

```
print(fruits[-1])   # cherry
```

Add & remove

```
fruits.append("mango")
```

```
fruits.remove("banana")
```

```
print(len(fruits))  # 3
```

Dictionaries { }

Key-value pairs

```
person = {  
    "name": "Alice",  
    "age": 30,  
    "city": "Mumbai"  
}
```

Access values

```
print(person["name"])    # Alice
```

```
print(person.get("age")) # 30
```

```
person["email"] = "a@b.com"
```

Control Flow

Conditionals

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"

print(f"Grade: {grade}") # Grade: B
```

Loops

```
# for loop
colors = ["red", "green", "blue"]
for color in colors:
    print(color)

# while loop
count = 0
while count < 3:
    print(f"Count: {count}")
    count += 1
```



Python uses indentation (4 spaces) instead of curly braces { } to define code blocks.

Functions

>>> Defining & Calling Functions

Define a function

```
def greet(name, greeting="Hello"):
    """Return a greeting message."""
    return f"{greeting}, {name}!"
```

Call the function

```
message = greet("Python")
print(message) # Hello, Python!
```

With custom greeting

```
print(greet("World", "Hey"))
# Hey, World!
```

Key Concepts

def

keyword to define a function

return

sends a value back to the caller

parameters

inputs the function accepts

defaults

optional values (greeting="Hello")

docstrings

"""describe what the function does"""

Modules & Imports

>>> Import Patterns

```
import os                # Import entire module
import json              # Built-in JSON module
from datetime import datetime  # Import specific item
from dotenv import load_dotenv  # Import from a package
import requests as req   # Import with alias
```

Built-in

os, json, sys,
datetime, math

pip install

requests, dotenv,
pandas, flask

Your own

my_module.py
→ import my_module

File Handling

Reading Files

```
# Safe file reading with "with"
with open("data.txt", "r") as file:
    content = file.read()
    print(content)

# File auto-closes after "with" block
```

Writing Files

```
# Write to a file
with open("output.txt", "w") as file:
    file.write("Hello!\n")
    file.write("Line two\n")

# "w" = write, "a" = append, "rb" = read
binary
```

Mode	Description	Use Case
"r"	Read (default)	Reading text files, configs
"w"	Write (overwrites)	Creating new files, saving results
"a"	Append	Adding to log files
"rb"	Read binary	Reading images, PDFs

Error Handling

```
>>> try / except / finally
try:
    number = int(input("Enter a number: "))
    result = 100 / number
    print(f"Result: {result}")

except ValueError:
    print("That is not a valid number!")

except ZeroDivisionError:
    print("Cannot divide by zero!")

except Exception as ex:
    print(f"Unexpected error: {ex}")

finally:
    print("Done – this always runs.")
```

try

Code that might cause an error

except

What to do when an error occurs

finally

Code that ALWAYS runs (cleanup)

Working with JSON

```
>>> JSON = JavaScript Object Notation (The Language of APIs)
```

```
import json
```

```
# Python dict → JSON string
```

```
data = {"name": "Alice", "skills": ["Python", "Azure"]}
```

```
json_string = json.dumps(data, indent=2)
```

```
print(json_string)
```

```
# JSON string → Python dict
```

```
parsed = json.loads(json_string)
```

```
print(parsed["skills"][0]) # Python
```

```
# Read JSON file
```

```
with open("config.json", "r") as f:
```

💡 `json.dumps()` = dict → string | `json.loads()` = string → dict | Think: s = string

REST APIs with Python

```
>>> Making HTTP Requests with the requests Library
```

```
import requests
```

```
# Define the request
```

```
url = "https://api.example.com/data"
```

```
headers = {
```

```
    "Content-Type": "application/json",
```

```
    "Authorization": "Bearer YOUR_KEY"
```

```
}
```

```
# Send GET request
```

```
response = requests.get(url, headers=headers)
```

```
# Check response
```

```
if response.status_code == 200:
```

```
    data = response.json() # Parse JSON response
```

```
    print(data)
```

Python Quick Reference

Concept	Syntax	Example
Variable	<code>name = value</code>	<code>city = "Mumbai"</code>
f-String	<code>f"text {var}"</code>	<code>f"Hello, {name}!"</code>
List	<code>[item1, item2]</code>	<code>langs = ["Python", "C#"]</code>
Dictionary	<code>{"key": "value"}</code>	<code>{"name": "Alice", "age": 30}</code>
Function	<code>def name(params):</code>	<code>def greet(name): return f"Hi {name}"</code>
for Loop	<code>for item in list:</code>	<code>for x in items: print(x)</code>
Try/Except	<code>try: ... except:</code>	<code>try: x=1/0 except: print("err")</code>
File Read	<code>with open("f") as f:</code>	<code>with open("d.txt") as f: f.read()</code>
Import	<code>import module</code>	<code>import json; from os import path</code>



You're Ready!

Now let's put Python to work.

- ✓ Practice with Jupyter Notebooks
- ✓ Explore Python documentation at docs.python.org
- ✓ Start building — the best way to learn is by doing